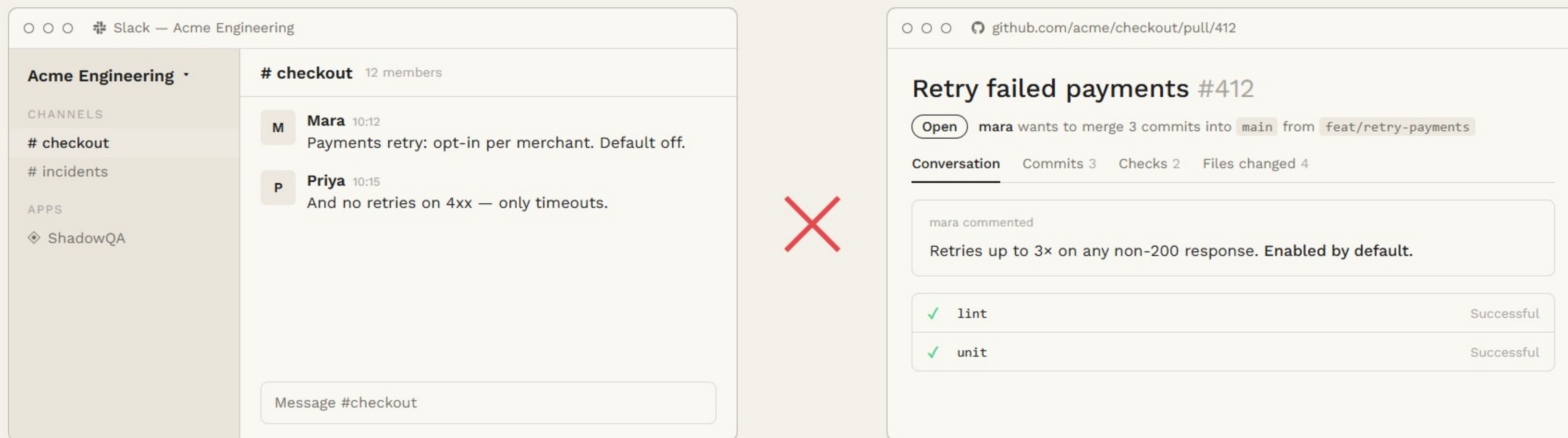




Context-aware QA — from the first message in Slack or Claude to the running app.

OBSERVE → PLAN → VERIFY → REPAIR

Decisions are made in Slack. Code ships from GitHub. Nothing reconciles the two.



Slack — Acme Engineering

Acme Engineering

CHANNELS

- # checkout
- # incidents

APPS

- ShadowQA

checkout 12 members

M Mara 10:12
Payments retry: opt-in per merchant. Default off.

P Priya 10:15
And no retries on 4xx — only timeouts.

Message #checkout

github.com/acme/checkout/pull/412

Retry failed payments #412

Open mara wants to merge 3 commits into `main` from `feat/retry-payments`

Conversation Commits 3 Checks 2 Files changed 4

mara commented

Retries up to 3x on any non-200 response. Enabled by default.

✓ lint	Successful
✓ unit	Successful

Every check is green. The pull request still contradicts the decision the team made an hour earlier — and nobody finds out until production.

Your Claude, your ChatGPT, your coding agent — none of them can see each other.

✱ Claude decides the design

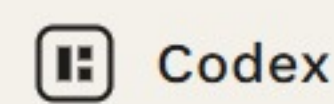
“Retry only on timeouts and 5xx. Never retry 4xx.”

🌀 ChatGPT writes the loop

“Here’s a loop that retries any failed request up to 5 times...”

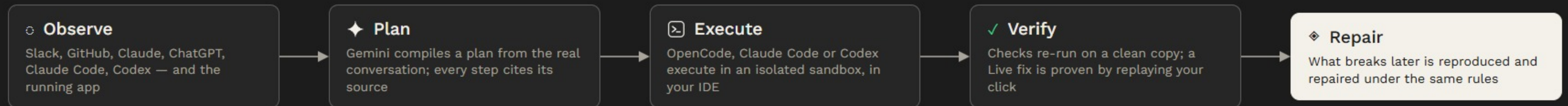
📄 The agent implements a third thing

With no memory of either conversation, in a repository neither has seen.



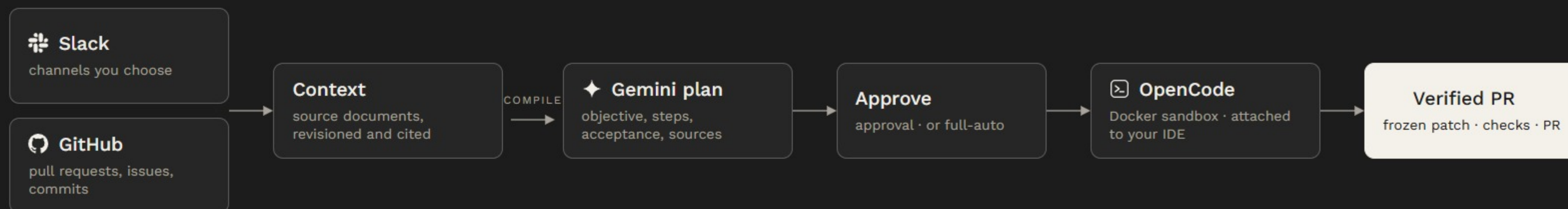
Context lives in six places. The code sees none of it.

ShadowQA watches where you already work, and turns it into verified code.



One desktop app, one findings list, one set of automation modes — for the plan, for the checks, and for the running application.

From a Slack thread to a verified pull request.



Connect

Slack · GitHub · mode · project, each verified live, from the app

Generate plan

context → repository inspection → Gemini plan

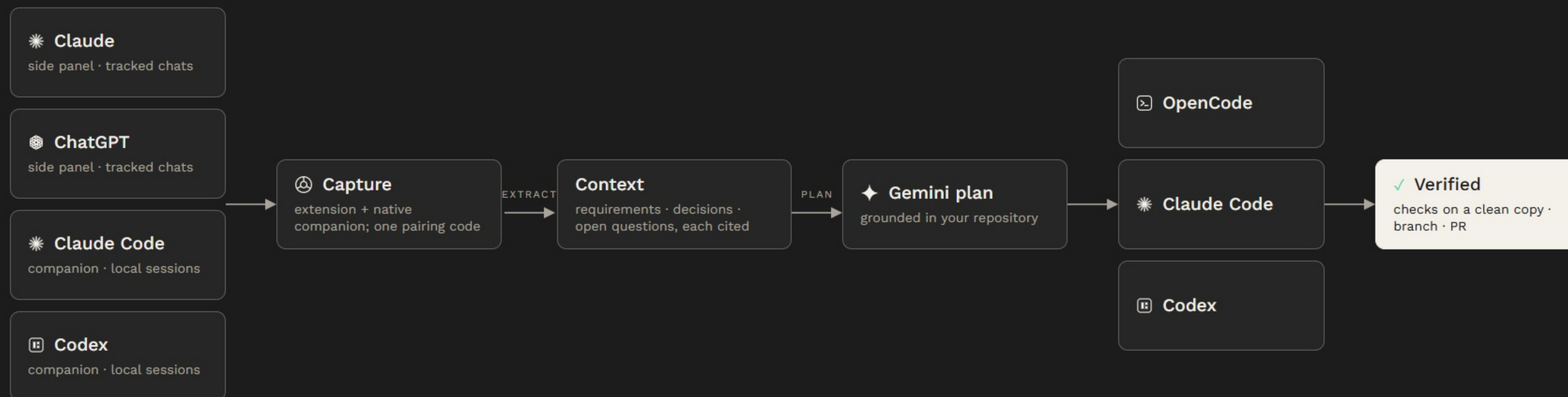
Approve

approves this exact digest, base SHA and command profile

Findings · Repair

what breaks later is reproduced, then repaired

Every conversation you choose to track becomes one context for one agent.



A question left open in the plan is shown, not hidden — and the agent runs in your IDE terminal, in a session you can step into.

Four modes decide how much ShadowQA may do on its own — everywhere, including Live.

observe

Watch and check only. ShadowQA never edits or publishes.

LIVE · diagnoses, never writes

approval

Ask first. Every plan and every repair waits for you.

LIVE · patch waits for Approve in the Live tab

auto-fix

Bounded automatic repair inside configured automatic paths; humans still merge.

LIVE · LOW-risk patches apply, still replay-verified

full-auto

Same bounded scope plus policy-gated merging. Requires an explicit merge policy.

LIVE · same restricted scope; merge policy in the service

SRC/CORE/CONTRACTS.TS

```
1 export const Mode = z.enum(["observe", "approval", "auto-fix", "full-auto"]);
```

LIVE/BACKEND/SHADOWQA/CONFIG.PY

```
1 MODE_TO_AUTONOMY = {
2     "observe": "observe",
3     "approval": "approve_all",
4     "auto-fix": "auto_low",
5     "full-auto": "auto_low",
6 }
```

Plans and checks cover the code. Live covers the running app.

localhost:3000/checkout

Lumen Supply Co.

CHECKOUT

Name

Mara Ellis

Address

14 Hollis Street

City

Bend, OR 97701

Card

4242 4242 4242 4242

Pay \$248.00

ORDER

Ridge Hatchet × 1

\$148.00

Waxed Canvas Pack × 1

\$100.00

Total

\$248.00

SHADOWQA LIVE

VERIFIED

Fixed and verified by replay

CHAIN

click Pay → POST /api/demo/payment → 500 → TypeError

ROOT CAUSE

Checkout read order.total; the amount must come from the cart

PATCH

4 lines · 2 files · risk LOW

REPLAY

✓ click Pay → 200 → order placed

Open PR

Undo

Capture	A tiny SDK (or the Chrome extension) watches clicks, requests, console and exceptions on localhost.
Correlate	The failing click, the request it caused and the exception it produced are joined into one incident — frontend and backend.
Diagnose	Claude finds the root cause, GPT as the fallback; source maps point at the real file and line.
Patch · grade	A minimal patch is proposed and risk-scored. The project mode decides whether it waits.
Validate · replay	Behind a checkpoint: your linters and tests run, then Live replays your exact interaction. Fail → automatic rollback.
Report	The incident is a finding on the Findings tab; Open pull request on the Live tab does the rest.



Built to be trusted with a repository.

Isolated execution

Every run happens in a locked-down Docker container with a fresh clone; the runner never touches your working tree.

Verified independently

Checks re-run on a clean copy and are compared to a baseline. The agent's own success claim is never trusted.

Never weakens a test

A repair that deletes or loosens a check is rejected by policy before it can be published.

Cited or rejected

A plan step must cite a real message or pull request; a plan that cites nothing real is refused.

Bounded repair

Two attempts per finding, a cooldown between them, then escalation to a human.

Approve the exact plan

Approval binds a digest, a base SHA and a command profile. Any drift invalidates it.

Live risk grading

Runtime patches are graded LOW / MEDIUM / HIGH; only LOW may ever apply without a person.

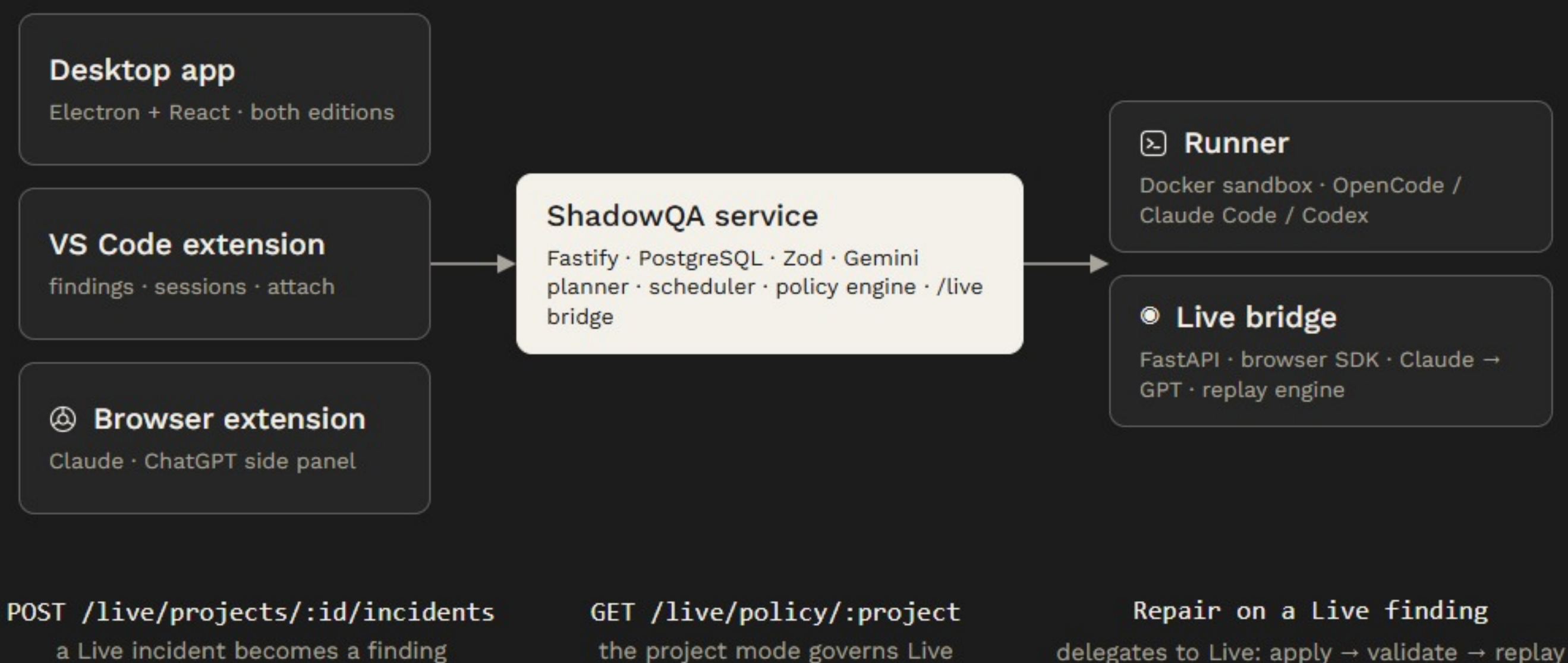
Kill switch

One switch in the desktop app's Admin tab stops and cancels everything, in every mode.

Works in your IDE

OpenCode, Claude Code and Codex sessions attach to your terminal; a VS Code extension shows findings inline.

Two runtimes, one vocabulary, one findings list.





Slack



GitHub



Gemini



Claude



OpenAI



OpenCode



Codex

github.com/BlueBrik1/shadowqa

OBSERVE → PLAN → VERIFY → REPAIR